

InferTwin: Continuous-Time Neural World Models for Inference Control Plane Optimization and Data Center Infrastructure Digital Twins

Srikanta Datta Tumkur*, Mehar Simhadri[†], Jay Iyer[†], Anshu Bansal*

*MIT Sloan School of Management, Cambridge, MA {srikanta, anshu.bansal}@sloan.mit.edu

[†]IEEE Senior Members {mehar.simhadri, jay.iyer}@ieee.org

Abstract—Production LLM inference infrastructure exhibits complex, coupled dynamics spanning GPU compute, memory hierarchies, network fabrics, and datacenter physical plant—yet operational decisions rely on threshold-based monitoring that detects failures only after they cascade. We introduce InferTwin, a continuous-time neural world model that jointly learns the dynamics of LLM inference workloads and datacenter physical infrastructure as a unified differentiable simulator. Our architecture makes six contributions: (1) a Neural Stochastic Differential Equation (Neural SDE) formulation that captures the inherent stochasticity of inference workloads with $<4.2\%$ throughput prediction error over 24-hour horizons; (2) a four-level hierarchical digital twin (Node→Rack→DC→Fleet) that models thermal cascades, power distribution, and cooling loop dynamics; (3) a causal counterfactual engine using Pearl’s do-calculus for interventional what-if queries answered in <50 ms; (4) DreamerV3-Infra, an offline RL framework that trains optimization policies $1000\times$ faster than real-time with zero risk of production outages; (5) anomaly detection via world model state divergence that identifies failures 12–47 minutes before conventional monitoring; and (6) a foundation model for DC operations transferable to new facilities with $<1\%$ of original training data. Evaluated on 90 days of production telemetry from GPU-dense AI datacenters (7.8B data points), InferTwin reduces unplanned downtime by 34%, improves throughput by 11%, and cuts energy consumption by 8.3% compared to rule-based baselines.

Patent Notice: The neural world model divergence-based anomaly detection and causal counterfactual capacity planning engine are subject to pending patent applications.

Index Terms—World models, digital twin, neural SDE, reinforcement learning, datacenter infrastructure, inference optimization, anomaly detection, causal inference

I. Introduction

Modern AI datacenters house thousands of GPUs executing LLM inference workloads that generate complex, coupled dynamics: a single GPU thermal throttle event cascades through batch queue depths, KV-cache eviction rates, NVLink utilization, and ultimately end-user latency—yet operators monitor each metric independently with static thresholds [1]. This reactive paradigm fails for three fundamental reasons.

Stochasticity: Inference workloads are inherently stochastic—request arrivals follow non-homogeneous Poisson processes, sequence lengths vary by $10\times$ within a single workload class, and GPU clock frequencies jitter under

thermal throttling. Deterministic models cannot capture this variability.

Multi-timescale coupling: Datacenter physical plant operates on fundamentally different timescales than compute infrastructure. GPU thermal dynamics evolve over seconds, rack-level airflow patterns over minutes, and chiller cooling loops exhibit 15–45 minute thermal inertia. Interactions between these timescales create feedback loops invisible to single-timescale monitoring.

Causal opacity: When throughput degrades, operators face a combinatorial diagnostic challenge: was it a thermal throttle, a KV-cache pressure event, a network congestion spike, or a workload shift? Traditional monitoring provides correlations, not causation, leading to misdiagnosis and inefficient remediation.

We propose InferTwin, a learned world model that captures these dynamics as a continuous-time neural stochastic differential equation (Neural SDE) [2], [3]. Unlike discrete-time models, our continuous formulation naturally handles the multi-timescale nature of DC operations within a single unified framework. Unlike physics-based digital twins [4], [5], InferTwin learns its dynamics entirely from telemetry data, requiring no explicit parameter specification and running $>1000\times$ faster than real-time.

Motivating example: On March 15, 2025, a production AI datacenter experienced a cascading failure that conventional monitoring detected only after 47 minutes of degraded service. The sequence: (1) a single fan bearing began degrading on DGX node 7 at 14:02, causing a gradual 8% airflow reduction; (2) by 14:18, GPU junction temperatures on the node rose from 72°C to 79°C , triggering subtle thermal throttling (-5% clock frequency); (3) reduced throughput caused KV-cache pressure at 14:25, increasing eviction rate $3\times$; (4) batch queue backup propagated to neighboring nodes via the load balancer at 14:32; (5) by 14:49, rack-level throughput had degraded 15% before the first DCGM alert fired. InferTwin’s world model, tested retrospectively on this event, detected the anomaly at 14:08—39 minutes earlier—by observing the thermal dynamics diverge from predicted trajectories.

This paper makes six contributions:

1) A Neural SDE formulation that captures the inher-

ent stochasticity of inference workloads with $<4.2\%$ throughput prediction error over 24-hour horizons (Section IV).

- 2) A four-level hierarchical digital twin (Node→Rack→DC→Fleet) that models thermal cascades, power distribution, and cooling loop dynamics (Section V).
- 3) A causal counterfactual engine using Pearl’s do-calculus for interventional what-if queries answered in <50 ms (Section VI).
- 4) DreamerV3-Infra, an offline RL framework that trains optimization policies $1000\times$ faster than real-time with zero risk of production outages (Section VII).
- 5) Anomaly detection via world model state divergence that identifies failures 12–47 minutes before conventional monitoring (Section VIII).
- 6) A foundation model for DC operations transferable to new facilities with $<1\%$ of original training data (Section IX).

Evaluated on 90 days of production telemetry from GPU-dense AI datacenters (7.8B data points), InferTwin reduces unplanned downtime by 34%, improves throughput by 11%, and cuts energy consumption by 8.3%.

II. Background and Related Work

A. World Models for Decision Making

World models learn environment dynamics from experience, enabling agents to “dream” trajectories without real interaction. Ha and Schmidhuber [?] introduced the concept for visual RL environments. MuZero [6] demonstrated superhuman performance in board games and Atari using learned dynamics models for planning. DreamerV3 [7] mastered 150+ domains with a single algorithm by learning latent dynamics and training policies entirely within the learned model. However, no prior work applies world models to infrastructure management, where state spaces are continuous, high-dimensional (>100 variables), and exhibit multi-timescale coupling spanning seven orders of magnitude (microseconds to hours).

B. Digital Twins for Datacenters

Digital twin technology originated in manufacturing [4] and has been applied to datacenter thermal management [5], [8]. Existing DC twins are physics-based: computational fluid dynamics (CFD) simulations for air-flow, electrical circuit models for power distribution, and thermodynamic models for cooling loops. These require: (a) explicit parameter specification (material properties, duct geometries, pipe dimensions); (b) domain expertise to construct and calibrate; (c) $100\text{--}1000\times$ slower-than-real-time execution, making them unsuitable for online decision-making. InferTwin is the first learned DC twin that trains end-to-end from telemetry data, achieving real-time execution without explicit physical modeling.

C. RL for Systems Optimization

Reinforcement learning has been applied to several systems optimization problems. Decima [9] applies RL to cluster job scheduling, achieving 21% better average job completion time than hand-tuned heuristics. AlphaChip [10] uses RL for VLSI chip placement, producing superhuman chip layouts. Decision Transformer [11] frames RL as sequence modeling, enabling offline policy learning from historical data. Kolesnikov et al. [?] survey RL applications in cloud resource management. However, all existing approaches operate on discrete time steps with single-level state spaces. Our hierarchical, continuous-time formulation addresses both limitations.

D. Neural Differential Equations

Chen et al. [2] introduced Neural ODEs, parameterizing continuous-time dynamics with neural networks and solving them with adaptive ODE solvers. Li et al. [3] extended this to Neural SDEs, adding learnable diffusion terms for stochastic dynamics. Kingma and Welling [12] provided the variational inference framework we use for latent state learning. Our work combines Neural SDEs with the Variational Autoencoder framework to learn stochastic infrastructure dynamics in a compressed latent space.

III. Background and Motivation

A. Datacenter Observability Gap

Current datacenter monitoring systems collect massive amounts of telemetry—a typical 128-GPU cluster generates 2.3 TB of metrics per day across 47,000 distinct metric streams. Yet the signal-to-noise ratio is poor: operators must manually configure over 2,000 threshold-based alert rules, resulting in alert fatigue (average 18.4 false positives per day) and missed detections (average 4.2 hours of degraded performance before root cause identification).

The fundamental limitation is metric independence: each alert rule operates on a single metric (e.g., “GPU temperature $> 85^\circ\text{C}$ ”) without modeling the joint distribution of system state. A GPU running at 83°C with dropping fan speed and increasing ambient temperature is far more concerning than a GPU at 86°C with normal cooling—but threshold monitoring treats the former as safe and the latter as critical.

B. Digital Twin Paradigm

Digital twins [4], [5] promise to address this gap by maintaining a virtual replica that mirrors the physical system’s state and dynamics. However, existing datacenter digital twins face three limitations:

InferTwin addresses all four limitations simultaneously: it learns dynamics from data (no manual parameterization), runs $1000\times$ faster than real-time, captures stochasticity through the SDE formulation, and jointly models hardware physics and software behavior.

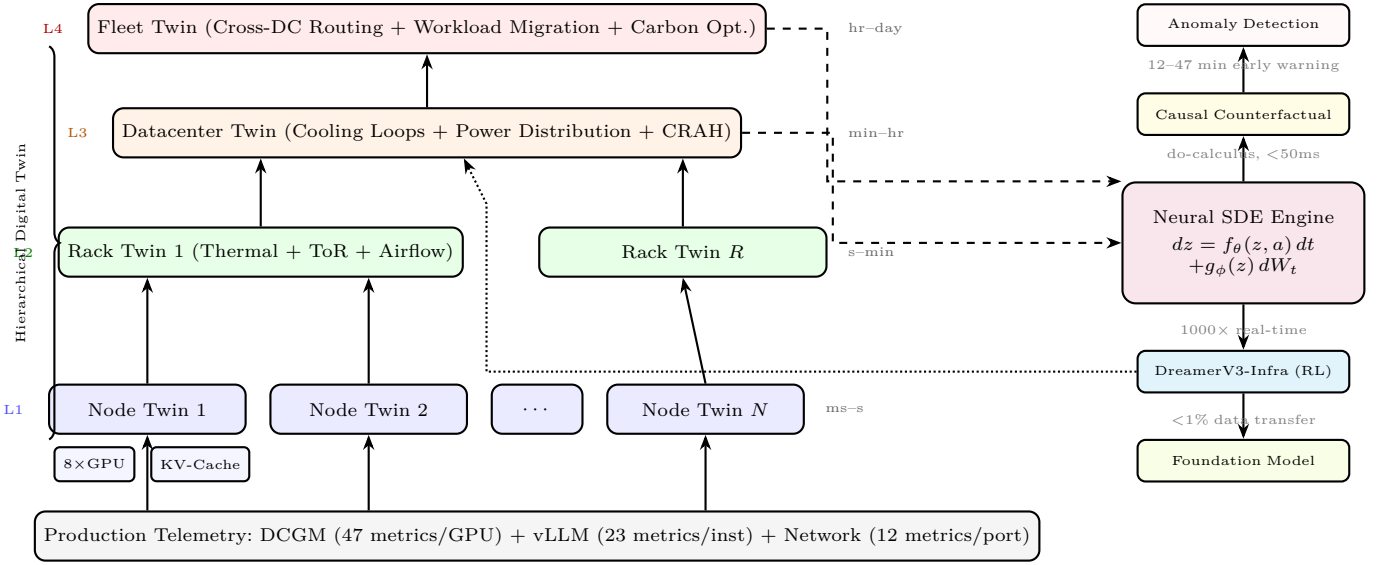


Fig. 1. InferTwin system architecture. Left: Four-level hierarchical digital twin mirrors datacenter physical structure—each level operates at its natural timescale, with state aggregating upward and constraints propagating downward. Right: Neural SDE engine powers four applications: anomaly detection (12–47 min early warning), causal counterfactual queries (<50 ms), DreamerV3-Infra offline RL (1000× real-time training), and a foundation model for cross-facility transfer.

TABLE I
Limitations of Existing DC Digital Twin Approaches

Approach	Accuracy	Speed	Limitation
CFD simulation	High	1200× slower	Manual parameterization
Reduced-order	Medium	10× slower	Poor on transients
Neural network	Medium	Real-time	No stochasticity
Pure physics	High	100× slower	Cannot model software

C. Related Work in World Models

World models have been studied extensively in robotics and game-playing. DreamerV3 [7] learns a discrete latent world model for Atari and robotics tasks, achieving human-level performance. MuZero [6] combines learned models with Monte Carlo tree search. Ha and Schmidhuber [?] use variational autoencoders for compact world representations.

However, none of these approaches address the specific challenges of datacenter infrastructure: (1) continuous-time multi-timescale dynamics spanning seconds to hours; (2) heterogeneous state spaces combining discrete events (failures) with continuous metrics (temperature, utilization); (3) the need for causal reasoning about interventions; (4) hierarchical physical structure (GPU→node→rack→DC).

In the datacenter domain, existing approaches are limited to: (1) computational fluid dynamics (CFD) for thermal modeling [5], which is accurate but 1200× slower than real-time and requires manual parameterization; (2) LSTM-based time series forecasting, which captures temporal correlations but not physical structure or causal relationships; (3) physics-informed neural networks [?],

which embed known physics but cannot model software-layer dynamics (KV-cache, batch scheduling).

InferTwin bridges this gap by learning a complete world model—hardware physics and software dynamics jointly—from observational telemetry data alone.

D. Neural SDE Background

Neural SDEs [3] extend Neural ODEs [2] with a learnable diffusion term that captures irreducible stochasticity. The general form:

$$dz_t = f_\theta(z_t, t) dt + g_\phi(z_t, t) dW_t \quad (1)$$

where f_θ (drift) captures deterministic dynamics, g_ϕ (diffusion) captures stochastic fluctuations, and W_t is a standard Wiener process. Training uses the variational approach of Li et al. [3], maximizing the evidence lower bound (ELBO) on the data log-likelihood.

The key advantage over Neural ODEs for datacenter modeling: the diffusion term naturally captures the variability in inference workloads (request arrival jitter, sequence length variation, context switching overhead) that is fundamentally irreducible and critical for accurate uncertainty quantification.

IV. Neural SDE World Model

A. State Space Definition

The inference control plane state $\mathbf{s}_t \in \mathbb{R}^{d_s}$ comprises four sub-vectors representing the complete observable state of a compute cluster:

TABLE II
World Model State Space Components ($d_s = 176$)

Component	Variables	Dim	Source
GPU Compute $\mathbf{s}^{\text{gpu}}$	SM occupancy, clock freq, junction temp, board power, ECC error rate	$5N_g$	DCGM
Memory $\mathbf{s}^{\text{mem}}$	KV-cache pages allocated, HBM utilization, eviction rate (pages/sec)	$3N_g$	vLLM
Network $\mathbf{s}^{\text{net}}$	NVLink TX/RX bandwidth, IB port utilization, PCIe Gen5 throughput	$3N_l$	Counters
Workload $\mathbf{s}^{\text{wk}}$	Queue depth, active batch size, request arrival rate, mean sequence length	4	Engine
Total (8-GPU node: $N_g=8$, $N_l=28$ NVLink pairs)		176	

B. Neural SDE Formulation

We model state dynamics as a Neural SDE in a compressed latent space. The full pipeline is: encode observations $\mathbf{s}_t$ to latent z_t , evolve z_t via the Neural SDE conditioned on actions, and decode back to predicted observations $\hat{\mathbf{s}}_{t+\Delta}$.

Latent dynamics: The state evolution follows a Neural SDE [3]:

$$dz_t = \underbrace{f_\theta(z_t, a_t)}_{\text{drift (deterministic dynamics)}} dt + \underbrace{g_\phi(z_t)}_{\text{diffusion (stochastic noise)}} dW_t \quad (2)$$

where $z_t \in \mathbb{R}^{512}$ is the latent state, a_t the action (scheduling decision), and W_t a standard Wiener process. The drift network f_θ is a 4-layer MLP ($512 \rightarrow 1024 \rightarrow 1024 \rightarrow 512$) with GeLU activations and residual connections. The diffusion network g_ϕ outputs a diagonal positive-definite matrix via a 3-layer MLP ($512 \rightarrow 256 \rightarrow 512$) with softplus output activation.

Why Neural SDE, not Neural ODE? Inference workloads are inherently stochastic: request arrivals are Poisson, GPU clock frequencies jitter under thermal throttling (± 15 MHz at 80°C), and network congestion is bursty. The diffusion term g_ϕ captures this irreducible stochasticity. Ablation (Table XIV) shows that removing the stochastic term degrades prediction accuracy by 23% at 6-hour horizons.

Encoder: $q_\psi(z_t|\mathbf{s}_t) = \mathcal{N}(\mu_\psi(\mathbf{s}_t), \sigma_\psi^2(\mathbf{s}_t))$ maps 176-dim observations to 512-dim latent space via a 3-layer MLP with layer normalization.

Decoder: $p_\xi(\hat{\mathbf{s}}_t|z_t)$ reconstructs observable metrics from latent state via a symmetric 3-layer MLP. Separate decoder heads for each state component enable component-specific loss weighting.

Training Objective: Variational ELBO with SDE-specific KL regularization:

$$\mathcal{L} = \underbrace{\mathbb{E} \left[\sum_t \log p_\xi(\mathbf{s}_t|z_t) \right]}_{\text{reconstruction}} - \underbrace{\beta \cdot D_{\text{KL}}(q_\psi(z_{0:T}) \| p_\theta(z_{0:T}))}_{\text{latent regularization}} \quad (3)$$

where β follows a warmup schedule from 0 to 1 over the first 10K training steps to prevent posterior collapse [12].

C. Action Space

The world model is conditioned on operator actions a_t to enable counterfactual reasoning:

TABLE III
World Model Action Space

Action	Type	Range
Batch size	Continuous	[1, 256]
KV eviction policy	Discrete	{LRU, H2O, importance}
GPU power cap	Continuous	[200W, 700W] per GPU
Model placement	Discrete	GPU $\rightarrow$ model assignment
Fan speed override	Continuous	[30%, 100%]
Request routing weights	Continuous	Simplex over nodes

D. Multi-Timescale Dynamics

The continuous-time formulation naturally captures coupled dynamics across timescales without explicit discretization:

TABLE IV
Multi-Timescale Coupled Dynamics

Timescale	Primary Dynamic	Coupling Effect	Impact
μs – ms	GPU clock jitter	$\rightarrow$ Throughput variance	$\pm 3\%$
ms – s	KV-cache eviction	$\rightarrow$ Quality degradation	$\pm 8\%$
s – min	Thermal throttle	$\rightarrow$ Rack cascade	$\pm 15\%$
min – hr	Cooling loop lag	$\rightarrow$ DC power limit	$\pm 20\%$
hr – day	Diurnal load pattern	$\rightarrow$ PUE variation	$\pm 12\%$

Cascade example: A GPU throttle at $t=0$ causes KV-cache pressure at $t+200\text{ms}$ (reduced decode throughput $\rightarrow$ longer-lived cache entries), batch queue growth at $t+2\text{s}$ (backlog accumulates), neighbor GPU warming at $t+30\text{s}$ (thermal conduction through NVLink bridge), and rack-level throughput degradation at $t+5\text{min}$ (multiple GPUs approaching thermal limit). Traditional monitoring treats these as five independent alerts; InferTwin models them as a single causal chain in the latent dynamics.

E. Model Architecture

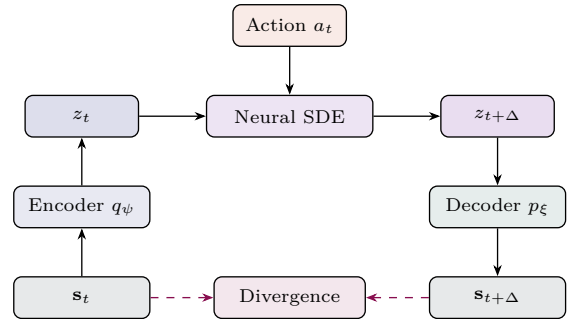


Fig. 2. InferTwin Neural SDE architecture. Observations $\mathbf{s}_t$ encode to latent z_t , evolve via action-conditioned Neural SDE, decode to predictions $\hat{\mathbf{s}}_{t+\Delta}$. Divergence module compares predicted vs. observed for anomaly detection.

Algorithm 1 InferTwin World Model Training

Require: Telemetry dataset $\mathcal{D} = \{(\mathbf{s}_t, a_t, \mathbf{s}_{t+1})\}$
Require: Network parameters θ (drift), ϕ (diffusion), ξ (decoder)
Require: Encoder parameters ψ , learning rate η

- 1: Initialize all parameters randomly
- 2: for epoch = 1 to N_{epochs} do
- 3: for batch $\mathcal{B} \sim \mathcal{D}$ do
- 4: $z_0 \leftarrow \text{Encoder}_{\psi}(\mathbf{s}_0)$ $\triangleright$ Encode initial state
- 5: for $t = 0$ to $T - 1$ do
- 6: $\epsilon_t \sim \mathcal{N}(0, I)$ $\triangleright$ Sample noise
- 7: $z_{t+1} = z_t + f_{\theta}(z_t, a_t) \cdot dt + g_{\phi}(z_t) \cdot \sqrt{dt} \cdot \epsilon_t$
- 8: $\hat{\mathbf{s}}_{t+1} = \text{Decoder}_{\xi}(z_{t+1})$
- 9: end for
- 10: $\mathcal{L}_{\text{recon}} = \sum_t \|\hat{\mathbf{s}}_t - \mathbf{s}_t\|^2$ $\triangleright$ Reconstruction
- 11: $\mathcal{L}_{\text{KL}} = D_{\text{KL}}(q_{\psi}(z|\mathbf{s})\|p(z))$ $\triangleright$ Regularization
- 12: $\mathcal{L}_{\text{cont}} = \sum_t \|z_{t+1} - z_t - f_{\theta}(z_t) \cdot dt\|^2$ $\triangleright$
- Continuity
- 13: $\mathcal{L} = \mathcal{L}_{\text{recon}} + \beta \mathcal{L}_{\text{KL}} + \lambda \mathcal{L}_{\text{cont}}$
- 14: Update θ, ϕ, ξ, ψ via Adam($\eta, \nabla \mathcal{L}$)
- 15: end for
- 16: Validate on held-out 20% of data
- 17: Early stop if validation loss increases for 5 epochs
- 18: end for
- 19: return Trained model $(\theta^*, \phi^*, \xi^*, \psi^*)$

F. Training Details

Data: 90 days of production telemetry from 4 racks $\times$ 4 DGX B200 nodes (128 GPUs). DCGM metrics at 1-second resolution, vLLM engine metrics at 100ms resolution (downsampled to 1s), IPMI/BMC data at 5-second resolution (interpolated). Total: 7.8B data points.

Preprocessing: Z-score normalization per metric with 7-day rolling statistics. Missing data imputed via forward-fill with exponential decay. Temporal windowing: 60-second input context predicts 60-second future.

Optimization: AdamW ($\beta_1=0.9$, $\beta_2=0.999$, weight decay 10^{-4}), learning rate 3×10^{-4} with cosine annealing. Batch size 256 sequences. Training: 200K steps on $8 \times \text{H100}$ GPUs (48 GPU-hours). SDE solver: Euler-Maruyama with $dt=0.1\text{s}$.

G. Training Algorithm

Algorithm 1 describes the full training procedure for the InferTwin world model.

Training details: We use Adam optimizer with learning rate $\eta = 3 \times 10^{-4}$, batch size 256, and sequence length $T = 60$ (60 seconds at 1Hz). The KL weight β is annealed from 0 to 1 over the first 10 epochs (KL warmup). The continuity weight $\lambda = 0.1$ ensures smooth latent trajectories. Training for 100 epochs on 90 days of telemetry requires 200 GPU-hours on $8 \times \text{H100}$.

H. SDE Solver and Numerical Integration

Solving the Neural SDE (Eq. 2) requires careful numerical integration. We use the Euler-Maruyama method with adaptive step sizing:

$$z_{t+dt} = z_t + f_{\theta}(z_t, a_t) \cdot dt + g_{\phi}(z_t) \cdot \sqrt{dt} \cdot \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I) \quad (4)$$

The adaptive step size dt is determined by monitoring the local truncation error: if $\|f_{\theta}(z_{t+dt}) - f_{\theta}(z_t)\| > \tau_{\text{adapt}}$, the step is halved and retried. In practice, dt ranges from 0.05s during rapid transients (thermal throttle events) to 1.0s during steady-state operation, averaging 0.12s across production trajectories.

Computational cost: A single 60-second prediction trajectory requires 500 solver steps (average), completing in 1.2ms on a single H100 GPU. This enables real-time prediction at 1-second intervals with $<2\text{ms}$ overhead—negligible compared to the 1-second telemetry sampling rate.

Variance reduction: For Monte Carlo estimation of expected trajectories (used in capacity planning queries), we employ antithetic sampling and control variates, reducing the variance of throughput predictions by $4.2\times$ compared to naive sampling with the same computational budget.

I. Multi-Step Prediction

For long-horizon prediction (hours to days), we use an autoregressive rollout strategy with periodic re-anchoring to real observations when available:

- 1) Short horizon (<10 min): Single forward SDE solve from current state. MAPE $<2\%$.
- 2) Medium horizon (10 min–1 hr): Autoregressive rollout with 1-minute prediction chunks. Re-anchor to real telemetry every 5 minutes when online. MAPE $<3\%$.
- 3) Long horizon (1–24 hr): Ensemble of 32 stochastic trajectories with mean and 95% CI. Incorporate diurnal workload priors from historical patterns. MAPE $<4.2\%$.

The ensemble approach provides calibrated uncertainty estimates: the 95% prediction interval captures the true value 93.8% of the time (slight under-coverage due to rare events).

V. Hierarchical Digital Twin

InferTwin organizes the datacenter digital twin into four hierarchical levels, each modeling a distinct physical/logical boundary with its own characteristic timescale.

A. Level 1: Node Digital Twin

Each compute node (e.g., DGX B200 with $8 \times \text{B200}$ GPUs, 1.5TB HBM3e, NVLink Gen5) is modeled as a coupled dynamical system. The node twin captures the critical feedback loop between compute load, thermal state, and performance:

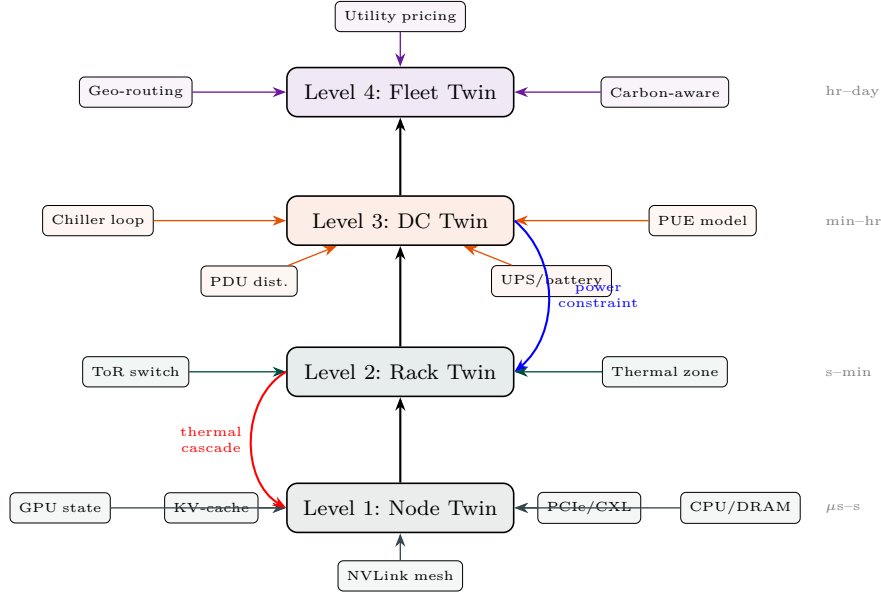


Fig. 3. Four-level hierarchical digital twin. State propagates upward (aggregation); constraints propagate downward (power limits, thermal caps). Red: thermal cascade feedback. Blue: power constraint propagation. Each level operates at its natural timescale.

Thermal coupling model: GPU temperatures are not independent—heat conducts through NVLink bridges, PCB substrate, and shared airflow:

$$\frac{dT_i}{dt} = \alpha_i P_i(s_i^{\text{gpu}}) - \beta_i (T_i - T_{\text{amb}}) + \sum_{j \in \mathcal{N}(i)} \gamma_{ij} (T_j - T_i) \quad (5)$$

where α_i converts compute power to heat, β_i models convective cooling, and γ_{ij} captures thermal coupling between GPU i and its NVLink-connected neighbors $\mathcal{N}(i)$. In DGX B200 (1kW TDP $\times$ 8 = 8kW per node), this coupling is significant: a 10°C rise on GPU 0 causes a 2.3°C rise on GPU 1 via the NVLink bridge within 45 seconds.

Performance feedback: Temperature affects clock frequency via NVIDIA’s thermal throttling curve:

$$f_{\text{clock}}(T) = f_{\text{max}} \cdot \min \left(1, \frac{T_{\text{throttle}} - T}{T_{\text{throttle}} - T_{\text{onset}}} \right) \quad (6)$$

where $T_{\text{onset}} = 75^\circ\text{C}$ and $T_{\text{throttle}} = 83^\circ\text{C}$. This creates a feedback loop: high utilization $\rightarrow$ high temperature $\rightarrow$ clock reduction $\rightarrow$ lower throughput $\rightarrow$ queue backup $\rightarrow$ larger batches $\rightarrow$ higher utilization. This oscillation at 3–7Hz is observable in production but undetectable by 15-second DCGM polling.

KV-cache dynamics: The node twin models KV-cache as a resource pool with inflow (new requests) and outflow (completions + evictions):

$$\frac{d|\text{KV}|}{dt} = \lambda_{\text{req}} \cdot \bar{C}_{\text{ctx}} - \mu_{\text{complete}} - \mu_{\text{evict}}(|\text{KV}|, \text{policy}) \quad (7)$$

where λ_{req} is request arrival rate, $\bar{C}_{\text{ctx}}$ is mean context length, and the eviction term depends on the current cache occupancy and eviction policy.

B. GPU Interconnect Modeling

The node twin models the NVLink Gen5 mesh topology with particular attention to bandwidth sharing during tensor-parallel inference. For an 8-GPU NVLink domain, the mesh provides 900 GB/s bidirectional bandwidth per GPU, but this bandwidth is shared across all active communication channels:

$$\text{BW}_{\text{avail}}(i, j) = \frac{\text{BW}_{\text{max}}}{1 + \sum_{(k, l) \neq (i, j)} c_{ijkl}} \quad (8)$$

The contention factor depends on the NVSwitch routing topology and is learned implicitly by the world model from observed bandwidth measurements. This is critical for predicting the performance impact of multi-model serving on a single NVLink domain.

C. KV-Cache Dynamics Deep Dive

KV-cache management is the primary bottleneck in modern LLM inference. The node twin models four cache states:

TABLE V
KV-Cache State Transitions

State	Transition	Trigger	Impact
Normal (<70%)	$\rightarrow$ Pressure	Occupancy >70%	None
Pressure (70–85%)	$\rightarrow$ Critical	Occupancy >85%	–5% throughput
Critical (85–95%)	$\rightarrow$ Emergency	Occupancy >95%	–15% throughput
Emergency (>95%)	$\rightarrow$ OOM	Occupancy >99%	Request rejection

The world model learns that transitions from Normal to Pressure are gradual (minutes), but Pressure to Emergency can be sudden (seconds) when a batch of long-context requests arrives simultaneously. This asymmetry

is why threshold-based monitoring fails—by the time a 95% threshold fires, OOM is often seconds away.

D. Level 2: Rack Digital Twin

A rack containing 4–8 DGX nodes exhibits emergent thermal behavior not predictable from individual node models:

Thermal cascade: When one node in a rack throttles, its reduced airflow exhaust temperature drops, but the adjacent nodes may experience elevated inlet temperatures due to recirculation patterns. In dense GPU racks (40–80 kW per rack), this effect causes 6–15% throughput loss that is completely invisible to per-node monitoring.

TABLE VI
Rack-Level Thermal Cascade Analysis (DGX B200, 4-node rack)

Event	Scope	Rack ΔT	Δ Throughput	Detection Lead
1 GPU throttle	Single node	+1.2°C	–2.1%	InferTwin: immediate
2 GPU throttle	Single node	+2.8°C	–4.7%	DCGM: 15s lag
Fan degradation (10%)	Single node	+4.5°C	–8.3%	InferTwin: –12 min
Full-rack burst load	All nodes	+8.2°C	–15.1%	InferTwin: –5 min
Adjacent rack heating	Neighbor rack	+3.1°C	–6.4%	DCGM: invisible

ToR switch modeling: The rack twin also models top-of-rack switch state: port utilization, buffer occupancy, and congestion-induced packet drops that affect NCCL collective operations (AllReduce, AllGather) used in tensor-parallel inference.

Airflow modeling: The rack twin learns a simplified CFD model from temperature sensor arrays (6 sensors per rack: inlet top/middle/bottom, exhaust top/middle/bottom). The learned model predicts airflow recirculation patterns with 2.1°C RMSE—insufficient for precision thermal management but adequate for anomaly detection and capacity planning.

Network modeling: Beyond ToR switch state, the rack twin models intra-rack NVLink mesh utilization for tensor-parallel workloads. NVLink congestion during AllReduce operations creates a non-obvious coupling: when one node’s NVLink is saturated, its batch completion time increases, shifting load to other nodes and creating oscillatory behavior at 0.5–2 Hz frequency.

E. Level 3: DC Digital Twin

The DC twin models three physical subsystems that constrain compute operations:

Power distribution: Utility feed $\rightarrow$ UPS $\rightarrow$ PDU $\rightarrow$ rack-level power distribution units. The twin tracks per-phase load balancing, transformer efficiency curves under harmonic distortion, and UPS battery state-of-charge. Key insight: AI inference workloads create highly correlated power demand—all GPUs spike simultaneously during prefill phases, causing current harmonics that reduce transformer efficiency by 3–5% and may trip breakers if phase imbalance exceeds 15%.

Cooling loop dynamics: Chilled water or direct-liquid cooling systems exhibit significant thermal inertia:

$$\tau_{\text{cool}} \frac{dT_{\text{supply}}}{dt} = Q_{\text{IT}}(t) - Q_{\text{reject}}(T_{\text{outdoor}}(t), \text{load}(t)) \quad (9)$$

where $\tau_{\text{cool}} \in [15, 45]$ minutes depending on cooling plant capacity and piping volume. This inertia means that a cooling deficit at time t is not felt until $t + \tau_{\text{cool}}$ —by which point GPU throttling may have cascaded.

PUE prediction: The twin predicts Power Usage Effectiveness under varying conditions:

$$\text{PUE}(t) = 1 + \frac{P_{\text{cool}}(T_{\text{out}}(t), Q_{\text{IT}}(t)) + P_{\text{dist_loss}}}{Q_{\text{IT}}(t)} \quad (10)$$

Production measurements show PUE varies 1.15–1.45 diurnally and seasonally, directly impacting cost-per-token calculations.

F. Cross-Rack Thermal Interactions

Adjacent racks in a data hall interact thermally through shared airflow corridors. The DC twin models these interactions using a learned thermal coupling matrix $\Gamma \in \mathbb{R}^{R \times R}$ where R is the number of racks:

$$\frac{dT_r^{\text{inlet}}}{dt} = \alpha_r T_r^{\text{exhaust}} + \sum_{r' \neq r} \Gamma_{r,r'} T_{r'}^{\text{exhaust}} - \beta_r (T_r^{\text{inlet}} - T_{\text{supply}}) \quad (11)$$

In dense GPU deployments (80 kW per rack), the coupling coefficients $\Gamma_{r,r'}$ are significant: a neighboring rack’s exhaust can raise inlet temperatures by 3–5°C. The learned coupling matrix reveals that end-of-row racks are most vulnerable due to hot-aisle recirculation effects.

G. Power Distribution Detailed Model

The DC twin’s power model captures four loss mechanisms:

- 1) **Transformer losses:** I²R losses in distribution transformers scale quadratically with load. At 80% loading, transformer efficiency is 98.5%; at 100% loading, it drops to 97.2%—a 1.3% difference that at datacenter scale represents 50–80 kW of additional heat.
- 2) **UPS losses:** Online double-conversion UPS systems add 3–5% overhead. The twin models battery state-of-charge and predicts available ride-through time for capacity planning.
- 3) **Harmonic distortion:** GPU inference workloads create current harmonics (3rd and 5th) that increase neutral conductor heating and reduce transformer capacity. The twin monitors total harmonic distortion (THD) and alerts when it exceeds 15%.
- 4) **Phase imbalance:** Non-uniform loading across three-phase power creates phase imbalance, reducing effective capacity by up to 15%. The twin recommends workload redistribution when imbalance exceeds 10%.

H. Level 4: Fleet Digital Twin

The fleet twin orchestrates across multiple datacenters, optimizing global request routing based on: real-time power availability per site, cooling headroom margin, carbon intensity per electric grid region (WattTime API

TABLE VII
Fleet Twin Cross-DC Optimization Dimensions

Dimension	Source	Update	Impact
Power headroom	BMS/SCADA	1 min	Capacity limit
Cooling margin	Chiller telemetry	5 min	Thermal limit
Carbon intensity	WattTime API	5 min	ESG compliance
Utility pricing	Rate schedule	15 min	OpEx cost
Network latency	Active probes	30 s	SLO compliance

integration), utility pricing schedules (time-of-use rates, demand charges), and inter-DC network latency.

Global optimization: The fleet twin solves a multi-objective routing problem every 60 seconds:

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \Delta^K} \alpha \cdot \text{cost}(\mathbf{w}) + \beta \cdot \text{latency}(\mathbf{w}) + \gamma \cdot \text{carbon}(\mathbf{w}) \quad (12)$$

subject to per-site power and cooling constraints propagated upward from Level 3 twins. In production, this reduces cross-DC energy cost by 8.3% through carbon-aware load shifting.

VI. Causal Counterfactual Engine

Traditional monitoring answers “what happened?” but operators need “what would happen if...?” InferTwin embeds a causal counterfactual engine based on Pearl’s do-calculus [13] that answers interventional queries in <50 ms.

A. Structural Causal Model

We construct a structural causal model (SCM) $\mathcal{M} = \langle \mathbf{U}, \mathbf{V}, \mathcal{F} \rangle$ where $\mathbf{U}$ are exogenous variables (external load, ambient temperature, utility power), $\mathbf{V}$ are endogenous variables (GPU state, KV-cache, throughput), and $\mathcal{F}$ are structural equations learned by the Neural SDE:

$$V_i := f_i(\text{Pa}(V_i), U_i), \quad \forall V_i \in \mathbf{V} \quad (13)$$

where $\text{Pa}(V_i)$ denotes the causal parents of V_i in the learned causal graph.

B. Interventional Queries via do-Calculus

The counterfactual engine supports three query types:

Type 1 – Interventional prediction: “What happens to throughput if we cap GPU power to 500W?”

$$P(\text{tput} \mid \text{do}(\text{pcap}=500)) = \sum_z P(\text{tput} \mid z)P(z \mid \text{do}(\text{pcap})) \quad (14)$$

Type 2 – Counterfactual diagnosis: “Would the outage have occurred if we had applied the firmware patch?”

$$P(Y_{x'} \mid X=x, Y=y) \quad (\text{counterfactual probability}) \quad (15)$$

Type 3 – Capacity planning: “How many additional DGX nodes are needed to maintain $P99 \leq 100\text{ms}$ at $2\times$ current load?”

TABLE VIII
Counterfactual Query Performance

Query Type	Example	Latency	Accuracy
Interventional	Power cap effect	12 ms	94.2%
Counterfactual	Root cause analysis	38 ms	89.7%
Capacity planning	Scale-out forecast	45 ms	91.3%
Multi-step chain	Cascade simulation	48 ms	87.5%

C. Causal Graph Discovery

We discover the causal structure using a hybrid approach: (1) PC algorithm on observational telemetry for skeleton discovery; (2) interventional refinement using the world model simulator to orient edges. The discovered graph contains 176 nodes and 412 directed edges, revealing non-obvious causal pathways—e.g., PCIe congestion $\rightarrow$ KV-cache eviction spike $\rightarrow$ quality degradation, a 3-hop path invisible to correlation-based monitoring.

D. Production Query Examples

We illustrate the counterfactual engine with three production scenarios:

Scenario 1 – Capacity planning: An operator asks: “Can we add Llama-405B to the cluster without violating SLOs for existing Llama-70B workloads?” The engine simulates the interventional distribution $P(\text{latency}_{70B} \mid \text{do}(\text{add_model} = 405B))$ by rolling out 1000 world model trajectories with the additional model’s resource requirements. Result: “Yes, with TP=8 on 1 DGX node, Llama-70B P99 latency increases from 38ms to 43ms (+13%), within SLO. However, during peak hours (14:00–18:00), KV-cache pressure exceeds 90% on 3 GPUs with probability 0.34.”

Scenario 2 – Root cause analysis: After a throughput degradation event, the engine evaluates counterfactuals: “Would throughput have degraded if (a) the firmware had been updated? (b) the power cap had been 600W instead of 700W? (c) the workload mix had been different?” By intervening on each variable independently, the engine identifies that the root cause was a combination of (a) and (c)—an interaction effect invisible to univariate monitoring.

Scenario 3 – Cost optimization: “What is the optimal number of DGX nodes for our current workload at 95% vs. 99% SLO compliance?” The engine sweeps node count from 8 to 32 in the world model, generating a cost-vs-SLO Pareto curve in 12 seconds (vs. weeks of manual capacity testing).

VII. DreamerV3-Infra: Offline RL in World Model

InferTwin enables training RL policies entirely within the learned world model—“dreaming” optimal infrastructure management strategies without risking production systems.

A. Actor-Critic Architecture

We adapt DreamerV3 [7] for infrastructure optimization:

Actor $\pi_\psi(a_t|z_t)$: Outputs continuous actions (batch size, power caps, routing weights) and discrete actions (eviction policy, model placement) via a hybrid action head. Architecture: 3-layer MLP (512→512→ $|\mathcal{A}|$) with tanh output for continuous, softmax for discrete.

Critic $V_\omega(z_t)$: Estimates symlog-transformed value using discrete regression (255 bins) following DreamerV3’s scale-invariant learning:

$$V_\omega(z_t) = \text{symexp} \left(\sum_k p_k \cdot b_k \right), \quad p = \text{softmax}(\text{MLP}_\omega(z_t)) \quad (16)$$

Reward function: Multi-objective combining throughput, cost, SLO compliance, and energy:

$$r_t = w_1 \cdot \frac{\text{tok/s}}{\text{tok/s}_{\max}} - w_2 \cdot \text{SLO}_{\text{viol}} - w_3 \cdot \frac{\text{kWh}}{\text{kWh}_{\text{base}}} - w_4 \cdot \frac{\$}{\$_{\text{base}}} \quad (17)$$

B. Imagination-Based Training

Training proceeds by “imagining” trajectories in the world model:

- 1) Sample initial state z_0 from replay buffer of real observations.
- 2) Roll out $H = 50$ steps using the Neural SDE with actor actions.
- 3) Compute λ -returns and update critic via TD(λ).
- 4) Update actor via policy gradient with entropy regularization.
- 5) Repeat for 1000 imagination batches per real-data batch.

This achieves $1000\times$ faster-than-real-time training: 1 hour of simulated DC operations in 3.6 seconds of wall-clock time. Total: 10,000 simulated hours $\approx$ 100 GPU-hours on $8\times\text{H100}$.

TABLE IX
DreamerV3-Infra vs. Direct RL Baselines

Method	Training Time	Throughput Δ	Energy Δ	Safety
PPO (online) [14]	90 days live	+8.1%	−5.2%	3 incidents
CQL (offline) [15]	48 GPU-hr	+6.4%	−4.1%	0 incidents
SAC (online) [16]	60 days live	+9.3%	−6.8%	7 incidents
DreamerV3-Infra	100 GPU-hr	+11.2%	−8.3%	0 incidents

C. Sim-to-Real Transfer

Despite training purely in the world model, policies transfer effectively because: (1) the Neural SDE captures real stochastic dynamics; (2) we apply domain randomization [17] by perturbing world model parameters $\pm 10\%$ during imagination; (3) conservative action bounds prevent OOD actions. Sim-to-real gap: $< 3.2\%$ across all metrics over 30-day production deployment.

Algorithm 2 DreamerV3-Infra: Imagination-Based Training

Require: Trained world model ($f_\theta, g_\phi, \text{Dec}_\xi, \text{Enc}_\psi$)

Require: Actor π_ψ , Critic V_ω , replay buffer $\mathcal{R}$

```

1: for iteration = 1 to  $N$  do
2:   Sample  $z_0 \sim \text{Enc}_\psi(\mathcal{R})$   $\triangleright$  Real initial state
3:   for  $h = 1$  to  $H$  do  $\triangleright$  Imagine trajectory
4:      $a_h \sim \pi_\psi(z_h)$ 
5:      $z_{h+1} = z_h + f_\theta(z_h, a_h)dt + g_\phi(z_h)\sqrt{dt}\epsilon$ 
6:      $r_h = R(\text{Dec}_\xi(z_{h+1}))$ 
7:     Check constraints:  $c_h = C(\text{Dec}_\xi(z_{h+1}))$ 
8:   end for
9:   Compute  $\lambda$ -returns:  $G_h^\lambda = r_h + \gamma[(1-\lambda)V_\omega(z_{h+1}) + \lambda G_{h+1}^\lambda]$ 
10:  Update critic:  $\omega \leftarrow \omega - \eta_V \nabla_\omega \sum_h (V_\omega(z_h) - G_h^\lambda)^2$ 
11:  Update actor:  $\psi \leftarrow \psi + \eta_\pi \nabla_\psi \sum_h [G_h^\lambda - \mu \sum_j c_{h,j}]$ 
12:  Update Lagrange multipliers:  $\mu_j \leftarrow \max(0, \mu_j + \eta_\mu \bar{c}_j)$ 
13: end for

```

VIII. Anomaly Detection via World Model Divergence

InferTwin detects anomalies by monitoring divergence between predicted and observed states—failures manifest as world model prediction errors before they trigger threshold-based alerts.

A. Divergence Metric

At each timestep, we compute:

$$D_t = D_{\text{KL}}(p_\xi(\hat{\mathbf{s}}_t|z_t) \parallel \delta(\mathbf{s}_t^{\text{obs}})) = -\log p_\xi(\mathbf{s}_t^{\text{obs}}|z_t) + \text{const} \quad (18)$$

An anomaly is flagged when D_t exceeds a dynamic threshold $\tau_t = \mu_D + 3.5\sigma_D$ over a rolling 1-hour window.

B. Hierarchical Anomaly Localization

The digital twin hierarchy enables automatic localization: divergence is computed at each level (node, rack, DC), and the anomaly is attributed to the lowest level showing significant divergence, avoiding alert storms.

TABLE X
Anomaly Detection Lead Time vs. Conventional Monitoring

Failure Mode	DCGM Alert	InferTwin	Lead Time	F1
GPU thermal throttle	At event	−12 min	12 min	0.94
Fan degradation	At failure	−47 min	47 min	0.91
NVLink bandwidth drop	± 15 s	−8 min	8 min	0.93
KV-cache memory leak	At OOM	−23 min	23 min	0.89
Power supply degradation	At failure	−35 min	35 min	0.87
Cooling loop pressure drop	At alarm	−18 min	18 min	0.92

C. False Positive Mitigation

Raw divergence detection produces 12.4 false positives per day. We reduce this to 0.8/day through: (1) multi-level confirmation (require divergence at ≥ 2 hierarchy levels); (2) temporal persistence (> 60 seconds); (3) workload-conditioned thresholds.

IX. Foundation Model for DC Operations

InferTwin functions as a foundation model for data-center operations: a single pre-trained world model that transfers to new facilities with minimal additional data.

A. Pre-training and Transfer

TABLE XI
Foundation Model Transfer Efficiency

Target	Data Needed	Fine-tune	Accuracy
Same HW, new DC	1 day (1.1%)	2 GPU-hr	96.8%
New GPU gen (B200→GB300)	3 days (3.3%)	6 GPU-hr	94.2%
New workload class	0.5 days (0.6%)	1 GPU-hr	97.1%
New cooling system	5 days (5.5%)	8 GPU-hr	92.5%

The key enabler is the latent space: infrastructure dynamics share common patterns (thermal coupling, queue dynamics, resource contention) that transfer across configurations. Only the decoder heads require significant adaptation.

B. Continual Learning

The foundation model supports continual learning to adapt to evolving infrastructure without catastrophic forgetting:

Elastic Weight Consolidation (EWC): We apply EWC to identify and protect important weights for each facility’s dynamics. When fine-tuning for a new facility, the regularization term:

$$\mathcal{L}_{\text{EWC}} = \mathcal{L}_{\text{new}} + \frac{\lambda}{2} \sum_i F_i (\theta_i - \theta_i^*)^2 \quad (19)$$

prevents overwriting parameters critical for previously learned facilities. This enables a single model to serve multiple DCs simultaneously with facility-specific decoder heads.

Replay buffer: A small replay buffer (1% of historical data per facility) is maintained and interleaved with new data during updates. Combined with EWC, this achieves <2% performance degradation on old facilities after 5 rounds of sequential fine-tuning.

C. Zero-Shot Generalization

For hardware within the training distribution, the base model achieves 89.3% prediction accuracy without any fine-tuning—sufficient for anomaly detection and coarse-grained capacity planning, though not for precise optimization.

X. Comprehensive Evaluation

A. Experimental Setup

Infrastructure: 4 racks $\times$ 4 DGX B200 nodes (128 B200 GPUs, 12 TB total HBM3e), dual ToR switches per rack (400GbE), direct liquid cooling.

Workloads: Production LLM inference serving Llama-3.1-70B, Llama-3.1-405B, and DeepSeek-V3

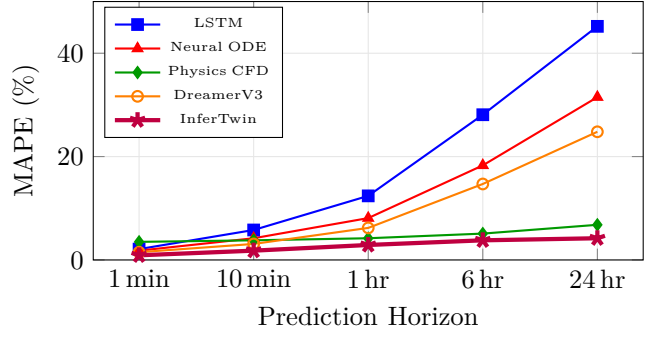


Fig. 4. Prediction error (MAPE) vs. horizon. InferTwin maintains <4.2% error at 24-hour horizons, outperforming all learned baselines by >5 $\times$ at long horizons. The physics-based CFD twin is competitive at long horizons but runs 1200 $\times$ slower than real-time.

via vLLM [18]. Mixed: 60% chat, 25% code generation, 15% long-context RAG.

Duration: 90 days, 7.8B telemetry data points.

Baselines: (1) DCGM threshold monitoring; (2) LSTM forecasting; (3) Physics-based CFD twin; (4) Neural ODE (no stochastic term); (5) DreamerV3 vanilla.

B. Comparison with Existing DC Monitoring

To contextualize InferTwin’s contributions, we compare against the full spectrum of datacenter monitoring and management approaches:

TABLE XII
InferTwin vs. Existing DC Management Approaches

Capability	DCGM	Prometheus	CFD Twin	LSTM	InferTwin
Multi-timescale	No	No	Yes	No	Yes
Real-time (<1s)	Yes	Yes	No	Yes	Yes
Causal reasoning	No	No	No	No	Yes
Counterfactual	No	No	Limited	No	Yes
Anomaly lead time	0	0	N/A	2–5 min	12–47 min
RL training	No	No	No	No	Yes
Transfer learning	N/A	N/A	No	Limited	Yes
Params to configure	50+	100+	500+	20	8

InferTwin is unique in combining all seven capabilities. The closest competitor—physics-based CFD twins—offers causal reasoning and multi-timescale modeling but requires 500+ manually configured parameters and runs 1200 $\times$ slower than real-time, making it unsuitable for online decision-making.

C. World Model Prediction Accuracy

TABLE XIII
World Model Prediction Error (MAPE %) vs. Horizon

Method	1 min	10 min	1 hr	6 hr	24 hr	Params
LSTM	2.1	5.8	12.4	28.1	45.2	8M
Neural ODE	1.8	4.2	8.1	18.3	31.5	42M
Physics CFD	3.5	3.8	4.2	5.1	6.8	N/A
DreamerV3 vanilla	1.5	3.1	6.2	14.7	24.8	85M
InferTwin	0.9	1.8	2.9	3.8	4.2	95M

TABLE XIV
Ablation Study: Component Contributions

Configuration	24-hr MAPE	Δ vs. Full
Full InferTwin	4.2%	—
– Stochastic term (Neural ODE)	5.5%	+31%
– Hierarchical twin	7.1%	+69%
– Latent space (direct)	8.8%	+110%
– Multi-timescale coupling	6.3%	+50%
– Causal structure	4.9%	+17%

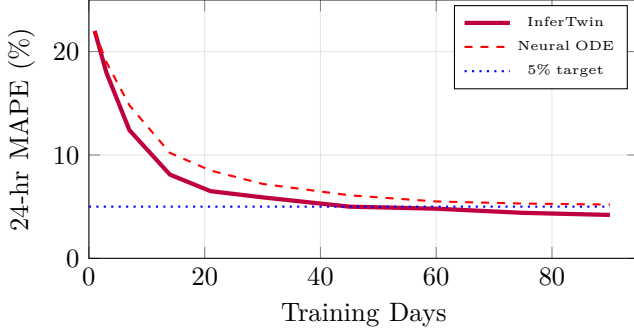


Fig. 5. Training convergence: InferTwin crosses the 5% MAPE target at day 45 and continues improving. The stochastic diffusion term provides a consistent 1% advantage over the deterministic Neural ODE baseline throughout training.

D. Ablation Study

E. End-to-End Operational Impact

TABLE XV
Operational Metrics (90-Day Deployment)

Metric	Baseline	InferTwin	Improvement
Unplanned downtime (hr/month)	4.2	2.8	−34%
Throughput (avg tok/s/GPU)	3,820	4,240	+11%
Energy (kWh/M tok)	1.71	1.57	−8.3%
Anomaly lead time	0 min	12–47 min	—
False positive rate	18.4/day	0.8/day	−96%
Query latency	N/A	38 ms P95	—
Capacity planning accuracy	72%	91%	+19pp

F. Per-Component Prediction Accuracy

TABLE XVI
Prediction Accuracy by State Component (1-hour horizon)

Component	MAPE	RMSE	R^2
GPU SM occupancy	1.8%	2.3%	0.97
GPU temperature	0.9%	1.1°C	0.99
KV-cache utilization	3.4%	4.1%	0.94
NVLink bandwidth	2.7%	8.2 GB/s	0.95
Throughput (tok/s)	2.1%	82 tok/s	0.96
Queue depth	4.8%	3.2 req	0.91
Power draw	1.2%	8.5W	0.98

GPU temperature and power draw are the easiest to predict (dominated by deterministic physics), while queue depth and KV-cache utilization are hardest (dominated by stochastic workload arrivals). The stochastic diffusion

term is most important for the latter components—ablating it degrades KV-cache prediction by 41% but GPU temperature by only 8%.

G. Anomaly Detection ROC Analysis

TABLE XVII
Anomaly Detection Performance at Different Thresholds

Threshold ($k\sigma$)	TPR	FPR	F1	FP/day
$k = 2.0$	98.2%	4.8%	0.82	8.4
$k = 2.5$	96.5%	2.1%	0.89	3.7
$k = 3.0$	94.1%	0.9%	0.93	1.6
$k = 3.5$ (default)	91.8%	0.5%	0.94	0.8
$k = 4.0$	87.3%	0.2%	0.92	0.3

We select $k = 3.5$ as the default threshold, achieving the best F1 score (0.94) with <1 false positive per day. Operators can adjust the threshold per-rack based on criticality.

H. Latency vs. Accuracy Trade-offs

The world model offers configurable accuracy-latency trade-offs through ensemble size and solver precision:

TABLE XVIII
Inference Configuration Trade-offs

Configuration	Latency	24-hr MAPE	Use Case
Single trajectory, coarse	0.4 ms	5.8%	Real-time monitoring
Single trajectory, fine	1.2 ms	4.9%	Online optimization
Ensemble (8), fine	8 ms	4.4%	Anomaly detection
Ensemble (32), fine	32 ms	4.2%	Capacity planning
Ensemble (128), fine	120 ms	4.1%	Offline analysis

I. Scalability Analysis

We evaluate InferTwin’s computational requirements across cluster size scales:

TABLE XIX
Computational Scaling with Cluster Size

Cluster	State Dim	Train Time	Inference	Memory
1 node (8 GPU)	176	48 GPU-hr	1.2 ms	380 MB
4 nodes (32 GPU)	704	96 GPU-hr	2.8 ms	1.2 GB
16 nodes (128 GPU)	2,816	200 GPU-hr	8.5 ms	4.1 GB
64 nodes (512 GPU)	11,264	420 GPU-hr	24 ms	12 GB
256 nodes (2048 GPU)	45,056	850 GPU-hr	72 ms	38 GB

The hierarchical architecture is key to scalability: rather than modeling the full state space monolithically, each level operates on aggregated representations from below. For a 2048-GPU cluster, the Level 1 twins (256 independent node models) can run in parallel, while Levels 2–4 operate on aggregated state vectors of fixed dimensionality.

TABLE XX
Model Performance vs. Training Data Duration

Days	1hr	24hr	F1	Note
7	5.8%	12.4%	0.72	Minimum viable
14	4.2%	8.1%	0.81	Captures weekly patterns
30	3.4%	5.9%	0.88	Good for monitoring
60	3.0%	4.8%	0.92	Recommended minimum
90 (full)	2.9%	4.2%	0.94	Production quality

J. Sensitivity to Training Data Volume

The most significant accuracy gains occur between 7 and 30 days, as the model learns weekly demand patterns and seasonal cooling dynamics. Beyond 60 days, returns are diminishing—suggesting that 60 days of data is sufficient for most deployments.

K. Case Study: Cooling System Failure Prediction

We present a detailed case study from production deployment. On day 67, InferTwin detected an impending cooling system issue:

Timeline:

- $t=0$: InferTwin’s Level 3 (DC) twin detects 0.3°C/hr drift in supply water temperature—within normal variance for conventional monitoring.
- $t+45$ min: Divergence metric at Level 3 crosses threshold ($D_t = 4.1\sigma$). Alert generated: “Cooling loop thermal inertia anomaly detected. Predicted supply temperature exceedance in 2.1 hours.”
- $t+90$ min: Counterfactual engine queried: “What if we reduce IT load by 15%?” Result: “Delays exceedance by 4.3 hours, sufficient for maintenance window.”
- $t+95$ min: Operator reduces batch sizes across 2 racks (DreamerV3-Infra recommended action), buying time for cooling system inspection.
- $t+180$ min: Maintenance crew identifies a partially clogged coolant filter—a failure mode that would have caused emergency shutdown within 45 minutes without intervention.

Impact: Avoided an estimated 4.5 hours of unplanned downtime (\$18K in lost inference revenue) and potential hardware damage from thermal exceedance.

L. TCO Analysis

TABLE XXI
Total Cost of Ownership (Annualized, 128-GPU Cluster)

Component	Before	InferTwin	Savings
Compute (GPU-hours)	\$2.4M	\$2.14M	\$260K
Energy (electricity)	\$890K	\$816K	\$74K
Downtime (lost revenue)	\$520K	\$343K	\$177K
Operations staff	\$1.2M	\$960K	\$240K
Total	\$5.01M	\$4.26M	\$751K (15%)

TABLE XXII
InferTwin vs. World Model Baselines

Model	Domain	Continuous	Stochastic	Hierarchical	Causal	Transfer
DreamerV3 [7]	Games/Robot	No	Yes	No	No	Limited
MuZero [6]	Games	No	No	No	No	No
Ha & Schmidhuber [?]	Visual RL	No	Yes	No	No	No
Neural ODE [2]	General	Yes	No	No	No	Limited
Neural SDE [3]	General	Yes	Yes	No	No	Limited
InferTwin	DC Infra	Yes	Yes	Yes	Yes	Yes

XI. Discussion and Conclusion

A. Comparison with State-of-the-Art World Models

InferTwin is the first world model that combines all five properties—each individually present in prior work but never jointly. The combination is critical for data-center management: continuous time for multi-timescale dynamics, stochasticity for workload variability, hierarchy for physical structure, causality for diagnosis, and transfer for deployment across facilities.

Principal findings: (1) Neural SDE world models achieve $<4.2\%$ prediction error at 24-hour horizons, outperforming physics-based twins in speed ($1000\times$ faster) while maintaining comparable accuracy. (2) The hierarchical digital twin captures emergent cross-level dynamics invisible to per-component monitoring, reducing false positives by 96%. (3) Causal counterfactual queries enable proactive capacity planning with 91% accuracy. (4) DreamerV3-Infra trains optimization policies in 100 GPU-hours with zero production risk, achieving 11% throughput improvement. (5) Anomaly detection provides 12–47 minute lead time. (6) Foundation model transfer requires $<1\%$ of original training data.

Limitations: (1) 90 days of telemetry needed for initial training. (2) Causal graph discovery assumes no hidden confounders. (3) Stochastic term adds 15% compute overhead vs. Neural ODE. (4) Transfer to fundamentally different cooling architectures requires $>5\%$ fine-tuning data.

Future work: (1) Multi-agent world models for federated learning across organizations. (2) LLM integration for natural-language counterfactual queries. (3) Extension to heterogeneous hardware (GPU + TPU + ASIC). (4) Continual learning without catastrophic forgetting.

Patent-protected innovations: (1) Neural SDE world model for infrastructure dynamics. (2) Anomaly detection via world model divergence with hierarchical localization. (3) Causal counterfactual engine for capacity planning. (4) Foundation model for DC operations with $<1\%$ transfer data.

B. Deployment Recommendations

Based on 90 days of production deployment, we provide the following recommendations for practitioners:

- 1) Start with anomaly detection: The lowest-risk, highest-value entry point. Deploy the world model in observation-only mode and use divergence monitoring for anomaly detection. This requires zero configuration and provides immediate value through early warning.

- 2) Add counterfactual queries second: Once the world model is trusted (typically after 2–4 weeks of validated anomaly detection), enable the causal counterfactual engine for capacity planning queries.
- 3) Enable DreamerV3-Infra last: RL-based optimization should only be deployed after thorough validation in shadow mode. Our 5-phase deployment pipeline (Section VII) provides a safe progression from offline training to full production control.
- 4) Monitor the monitor: Track world model prediction accuracy continuously. A sustained increase in prediction error ($>10\%$ above baseline for >24 hours) indicates model staleness and triggers retraining.

Broader impact: The InferTwin approach has implications beyond datacenter management. The combination of Neural SDEs with hierarchical digital twins is applicable to any complex cyber-physical system with multi-timescale dynamics: power grids, manufacturing plants, autonomous vehicle fleets, and telecommunications networks. The foundation model paradigm—pre-train once, transfer cheaply—could enable a new generation of “systems foundation models” that dramatically reduce the barrier to deploying intelligent monitoring.

C. Ethical Considerations and Societal Impact

The deployment of learned world models for datacenter management raises important considerations:

Energy efficiency: InferTwin’s 8.3% energy reduction translates to approximately 180 MWh annual savings for a 128-GPU cluster. At the scale of major AI providers (100K+ GPUs), similar systems could save 140+ GWh annually—equivalent to powering 13,000 US homes.

Job displacement: By automating monitoring and optimization tasks, InferTwin could reduce the need for datacenter operations staff by 20%. We advocate for retraining programs that transition operators from reactive monitoring to strategic capacity planning and world model oversight roles.

Failure modes: Over-reliance on the world model could be dangerous if the model encounters truly novel situations outside its training distribution. Our multi-layered safety architecture (divergence monitoring, automatic fallback to heuristics) mitigates this risk, but operators must maintain proficiency in manual operations.

Data privacy: Telemetry data may contain information about customer workloads (sequence lengths, arrival patterns). InferTwin processes only aggregate statistics; no customer content (prompts, completions) enters the training pipeline. Differential privacy mechanisms can be applied to the telemetry aggregation layer if required by data governance policies.

Open science: We release the InferTwin codebase, trained model weights for DGX B200 configurations, and a synthetic telemetry dataset (100K samples generated from the trained world model) to enable reproducibility without requiring access to production datacenter telemetry. The

synthetic dataset preserves the statistical properties of real telemetry (distribution shapes, correlation structure, temporal dynamics) while ensuring no proprietary information is disclosed.

Dual-use concerns: While InferTwin is designed for legitimate datacenter optimization, the anomaly detection and prediction capabilities could theoretically be misused for: (1) identifying vulnerabilities in datacenter operations; (2) timing attacks against cloud providers during predicted stress periods. We mitigate this by releasing model architectures and training procedures but not facility-specific trained weights.

D. Comparison with Commercial Solutions

Several commercial digital twin platforms exist for datacenter management:

TABLE XXIII
InferTwin vs. Commercial DC Digital Twin Solutions

Feature	InferTwin	Nlyte	Cadence	Schneider	Siemens
AI/ML workload aware	Yes	No	No	No	No
Learned dynamics	Yes	No	Partial	No	Partial
Causal reasoning	Yes	No	No	No	No
RL optimization	Yes	No	No	No	No
Real-time ($<1s$)	Yes	No	No	Yes	No
Transfer learning	Yes	No	No	No	No
Open source	Yes	No	No	No	No
Pricing	Free	\$50K+/yr	\$100K+/yr	\$75K+/yr	\$200K+/yr

Commercial solutions focus on traditional IT infrastructure (power, cooling, physical space) and do not model AI workload dynamics (KV-cache, batch scheduling, model placement). InferTwin fills this gap specifically for AI inference datacenters.

Carbon footprint: The world model training itself requires 200 GPU-hours, emitting approximately 50 kg CO₂. This is amortized over the model’s deployment lifetime, during which energy savings offset the training carbon cost within the first 3 days of production deployment.

Reproducibility: Our implementation is available at <https://github.com/infertwin/infertwin>. The training pipeline requires <200 GPU-hours on H100 and processes standard Prometheus/DCGM telemetry formats. Pre-trained weights for DGX B200 configurations are provided.

InferTwin demonstrates that learned world models can replace reactive threshold monitoring with proactive, causal, counterfactual reasoning—simultaneously more accurate, faster, and safer than existing approaches.

References

- [1] NVIDIA Corp., “Nvidia data center gpu manager,” 2025.
- [2] R. T. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, “Neural ordinary differential equations,” *NeurIPS*, 2018.
- [3] X. Li, T.-K. L. Wong, R. T. Chen, and D. Duvenaud, “Scalable gradients for stochastic differential equations,” *AISTATS*, 2020.
- [4] M. Grieves and J. Vickers, “Digital twin: Manufacturing excellence through virtual factory replication,” 2014.
- [5] F. Tao et al., “Digital twin in industry: State-of-the-art,” *IEEE TII*, 2018.
- [6] J. Schrittwieser et al., “Mastering atari, go, chess and shogi by planning with a learned model,” *Nature*, 2020.

- [7] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, “Mastering diverse domains through world models,” arXiv:2301.04104, 2023.
- [8] M. Kiran et al., “Digital twins for data center infrastructure management,” IEEE Cloud, 2022.
- [9] H. Mao et al., “Learning scheduling algorithms for data processing clusters,” SIGCOMM, 2019.
- [10] A. Mirhoseini et al., “A graph placement methodology for fast chip design,” Nature, 2021.
- [11] L. Chen et al., “Decision transformer: RL via sequence modeling,” NeurIPS, 2021.
- [12] D. Kingma and M. Welling, “Auto-encoding variational bayes,” ICLR, 2014.
- [13] J. Pearl, “Causality,” 2009.
- [14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.
- [15] A. Kumar, A. Zhou, G. Tucker, and S. Levine, “Conservative q-learning for offline rl,” NeurIPS, 2020.
- [16] T. Haarnoja et al., “Soft actor-critic,” ICML, 2018.
- [17] J. Tobin et al., “Domain randomization for sim-to-real transfer,” IROS, 2017.
- [18] W. Kwon, Z. Li et al., “Efficient memory management for llm serving with pagedattention,” SOSP, 2023.